Hands-On Generative AI

Language Generation Tasks & Prompting Techniques

Day 2 - Session 3

KavinSchool

What We'll Cover

1. Generative tasks: completion, dialogue, summarization, code
2. The anatomy of a good prompt
3. Techniques: zero-shot, few-shot, chain-of-thought, roles, structured output
4. Responsible prompting: injection, leaking, hijacking
5. Two local demos: prompting techniques, prompt injection
6. Lab 2.3: prompting for summarization, code, and labeling

Generative Tasks

What we ask LLMs to produce

KavinSchool

Common Generative Tasks

Task	What the model does
Completion	Continue or fill in text
Dialogue	Hold a multi-turn conversation
Summarization	Condense long text to key points
Code generation	Write or explain code

The same model handles all of these. The prompt decides which one you get.

One Model, Many Tasks

You do not train a new model per task. You describe the task in the prompt.

- "Summarize this in three bullets" gives summarization
- "Write a Python function that..." gives code generation
- "Label each review as positive or negative" gives classification

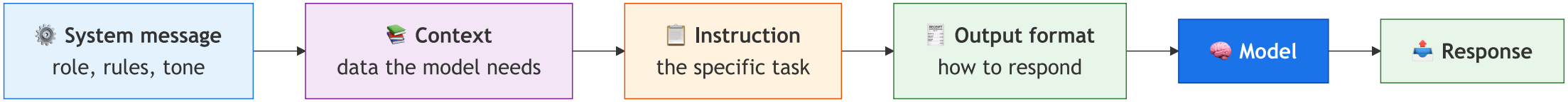
Prompting is how you steer a general model to a specific job.

Anatomy of a Prompt

The pieces of a good instruction

KavinSchool

What Goes into a Prompt



System vs User Messages

- **System message:** sets the role, rules, and tone. Persistent across the chat.
- **User message:** the specific request or data for this turn.

```
messages = [
    {"role": "system", "content": "You are a concise technical writer."},
    {"role": "user", "content": "Explain what a token is in two sentences."},
]
```

Be Specific

Vague prompts give vague answers. Say exactly what you want.

- State the format (bullets, JSON, a table)
- Give constraints (length, tone, audience)
- Provide the data the model needs

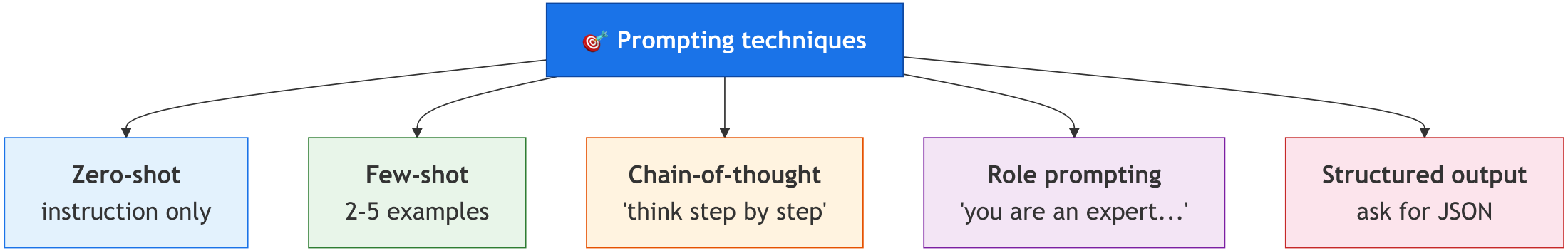
Specific instructions are the cheapest way to improve output.

Prompting Techniques

From zero-shot to chain-of-thought

KavinSchool

Five Techniques



Zero-Shot

Give the task with no examples. The model relies on what it already knows.

```
system = "Classify the sentiment as POSITIVE, NEGATIVE, or NEUTRAL."
user = "The update completely broke my workflow."
# Output: NEGATIVE
```

Try zero-shot first. It works well for standard tasks and costs the fewest tokens.

Few-Shot

Provide 2 to 5 examples to teach the model your pattern and format.

```
messages = [
    {"role": "system", "content": "Classify the sentiment."},
    {"role": "user", "content": "Best purchase ever."},
    {"role": "assistant", "content": "POSITIVE"},
    {"role": "user", "content": "Broke after one day."},
    {"role": "assistant", "content": "NEGATIVE"},
    {"role": "user", "content": "The battery could be better but I like it."},
]
```

Few-Shot: Best Practices

- Use 3 to 5 examples, enough to show the pattern
- Cover edge cases the model might get wrong
- Balance the categories, do not show only positives
- Order matters: put strong examples first and last

Few-shot teaches format, style, and how to handle tricky inputs.

Chain-of-Thought

Ask the model to reason step by step before answering.

```
Q: A store has 15 apples. 8 are sold, then 3 are delivered.
   How many remain? Think step by step.
A: Start 15, minus 8 is 7, plus 3 is 10. Answer: 10.
```

Use it for math, logic, and multi-step problems. Skip it for simple classification.

Role Prompting

Assign a persona to shape depth, style, and focus.

```
system = ("You are a senior Python developer who values clean code, "
          "type hints, and clear error messages.")
```

The same question answered as an expert reads very differently from a generic answer.

Structured Output

Ask for a specific format so your code can parse the result.

```
system = ("Extract the fields and reply with JSON only: "
          '{"name": str, "amount": number, "due_date": str}')
```

Structured output turns an LLM into a reliable component in a larger program.

Demo: Prompting Techniques

Run `day2/demos/demo_prompting_techniques.py`.

- Run zero-shot, few-shot, and chain-of-thought on a small local model
- See few-shot examples lock in the output format
- Watch "think step by step" turn a wrong direct answer into a correct one

> Runs locally, no API key. The technique lives in the prompt, not the provider.

Controlling Output

API parameters for fine-tuning behavior

KavinSchool

Key API Parameters

Parameter	What it does	Recommended
`temperature`	Controls creativity. Low = focused, High = random	`0.0` for code/JSON, `0.7` for chat
`top_p`	Alternative to temperature (nucleus sampling)	Use one or the other, not both
`max_tokens`	Hard cap on response length	Always set to prevent runaway costs
`stop`	String sequences that halt generation immediately	Great for forcing clean code blocks
`seed`	Attempts to make outputs deterministic	Useful for testing and demos

Streaming Responses

For production apps, waiting 5-10 seconds for a full answer is a poor user experience.

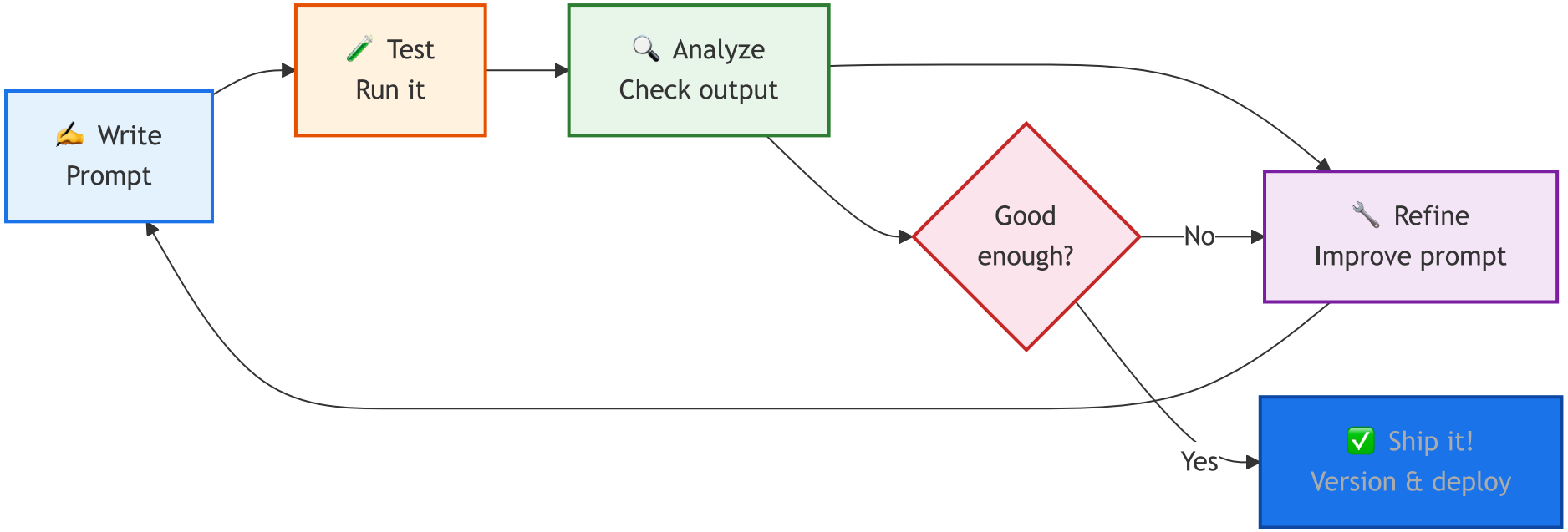
- **Streaming** sends tokens as they are generated
- The user sees text typing out in real-time (~200ms to first token)
- Essential for chatbots, long-form content, and high-latency models

Prompt Iteration

Treat Prompts Like Code

KavinSchool

The Iteration Loop



Refining Prompts

A prompt is never "done" on the first try. Plan for **3-5 iterations** minimum.

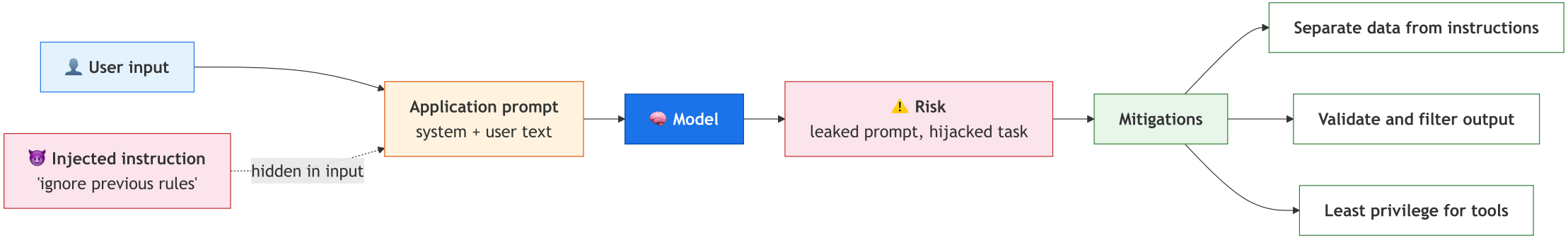
Symptom	Fix
Wrong format	Add stricter format instructions + a few-shot example
Too verbose	Add a word/sentence limit constraint
Hallucination	Add "Only use information from the provided text"
Inconsistent	Lower temperature, add balanced few-shot examples
Off-topic	Separate data from instructions using delimiters

Responsible Prompting

Prompt injection, leaking, and hijacking

KavinSchool

The Risk



An attacker hides an instruction inside user input, and the application prompt passes it straight to the model along with the real system and user text. The model cannot tell your instructions from the attacker's, so it may leak the system prompt or hijack the task. The three mitigations on the right cut this off at different points in the flow.

Three Failure Modes

- **Prompt injection:** user input carries instructions that override yours
- **Prompt leaking:** the model reveals your hidden system prompt
- **Prompt hijacking:** input redirects the model to a different task

These matter most when input comes from untrusted users or fetched web content.

Mitigations

- Keep instructions separate from user-supplied data
- Treat retrieved and user text as data, never as commands
- Validate and filter the output before acting on it
- Give tools the least privilege they need

No single fix is complete. Layer these defenses.

Demo: Prompt Injection

Run `day2/demos/demo-prompt-injection.py`.

- Attack a naive prompt with injection, leaking, and hijacking payloads
- Watch the model leak its system prompt and abandon its task
- Re-run against a hardened prompt that fences off data and validates output

> Runs locally, no API key. Keep instructions separate from untrusted input.

Breakout Lab 2.3

Prompting for summarization, code, and labeling

KavinSchool

Lab 2.3: Prompting Techniques

Open `day2/lab2_3_prompting/start.ipynb` in Colab.

- Summarize text with zero-shot, then refine with role and format
- Generate a Python function from a spec
- Label support tickets with zero-shot, then improve with few-shot
- Compare outputs and note what each technique changed

> Stretch goal: add chain-of-thought, or ask for JSON output and parse it.

Key Takeaways

1. One general model handles many tasks; the prompt picks the task
2. A good prompt sets role, context, instruction, and output format
3. Start zero-shot, add few-shot examples, use chain-of-thought for reasoning
4. Structured output makes an LLM a dependable part of your code
5. Treat untrusted input as data and layer defenses against injection

Questions?

Next: Agentic AI

Evening break: 15 minutes

KavinSchool